

Física com Python

Sidney Cerqueira

2026

Sumário

0	Python: ambiente e fundamentos	1
0.1	O papel do Python neste livro	1
0.2	Interpretador, programa e script	1
0.3	Onde escrever e executar Python	2
0.4	O primeiro programa	3
0.5	Valores, variáveis e atribuição	3
0.6	Tipos fundamentais	4
0.7	Operadores aritméticos	4
0.8	Apresentação de resultados	5
0.9	Funções	5
0.10	Módulos e bibliotecas	6
0.11	Arquivos, pastas e caminhos com pathlib	7
0.12	Primeiros gráficos com Matplotlib	8
0.12.1	Um primeiro gráfico a partir de dados	9
0.12.2	Amostragem de um intervalo com NumPy	10
0.12.3	Eixos, estilos, legenda, grade e anotação	11
0.12.4	Mais de um sistema de eixos	13
0.12.5	Linhas de referência, marcas, proporção e formas	13
0.12.6	Critérios para um gráfico científico	14
0.13	Ambiente virtual e dependências	15
0.14	Erros como informação	15
0.15	Critério de validação	15
1	Ferramentas matemáticas para a Física	17
1.1	Por que começar pela matemática?	17
1.2	Potências e raízes	17
1.2.1	Potências em Python	18
1.3	Notação científica e ordem de grandeza	20
1.3.1	Operações com notação científica	20
1.3.2	Notação científica em Python	21
1.3.3	Ordem de grandeza	21

1.3.4	Estimativas físicas	22
1.4	Proporcionalidade e leis de escala	23
1.4.1	Proporcionalidade direta	23
1.4.2	Proporcionalidade com uma potência	24
1.4.3	Proporcionalidade inversa	25
1.4.4	Proporcionalidade conjunta	26
1.4.5	Isolamento de variáveis	26
1.4.6	Limites de uma proporcionalidade	27
1.5	Logaritmos e escalas logarítmicas	28
1.5.1	Bases mais utilizadas	28
1.5.2	Propriedades fundamentais	29
1.5.3	Mudança de base	29
1.5.4	Logaritmos e leis de potência	29
1.5.5	Escalas logarítmicas	30
1.5.6	Argumentos dimensionais	31
1.5.7	Logaritmos e exponenciais como operações inversas	31
1.5.8	Logaritmos em Python	32
1.6	Ângulos e trigonometria	34
1.6.1	Seno e cosseno em Python	37
1.7	Funções e gráficos	40
1.7.1	Representação de funções em Python	41
1.7.2	Relação linear	43
1.7.3	Leis de potência e relações inversas	44
1.7.4	Comportamento exponencial	44
1.8	Derivada: taxa de variação instantânea	44
1.8.1	Derivada analítica e derivada numérica	45
1.8.2	Regras de derivação mais utilizadas	46
1.9	Integral: acumulação	49
1.9.1	Integral analítica e integração numérica	49
1.9.2	Regras de integração mais utilizadas	51
1.10	Máximos, mínimos e interpretação física	52
1.11	Estratégia para resolver problemas	53
1.12	Síntese	54
1.13	Exercícios propostos	54
	Resoluções das atividades propostas	57
	Capítulo 1 — Ferramentas matemáticas para a Física	57

Capítulo 0

Python: ambiente e fundamentos

0.1 O papel do Python neste livro

Python é uma linguagem de programação de propósito geral. Neste livro, ela será usada como ferramenta de investigação: realizaremos cálculos, analisaremos dados, construiremos gráficos, desenharemos vetores e simularemos modelos físicos.

O código não substitui a formulação matemática nem a interpretação física. Ele permite executar um procedimento de forma reproduzível e explorar situações que seriam trabalhosas manualmente. Um resultado numérico só se torna fisicamente útil depois que suas unidades, hipóteses e limites são examinados.

Adotaremos o **Python 3.12**. Versões posteriores da série 3 provavelmente executarão a maior parte dos exemplos, mas a versão 3.12 será nossa referência de desenvolvimento e validação. Fixar uma versão reduz diferenças de comportamento entre computadores e facilita a reprodução dos resultados.

0.2 Interpretador, programa e script

O interpretador Python é o programa que lê e executa instruções escritas na linguagem Python. Uma instrução pode ser digitada em uma sessão interativa ou armazenada em um arquivo de texto com extensão **.py**.

Um arquivo como

```
primeiro_programa.py
```

é chamado de *script*. No terminal integrado do VS Code, ele pode ser executado com

```
python primeiro_programa.py
```

Neste projeto, o interpretador do ambiente virtual pode ser selecionado diretamente pelo VS Code. Isso garante que o editor utilize a mesma versão de Python e as mesmas bibliotecas usadas na validação do livro.

0.3 Onde escrever e executar Python

Python não está vinculado a um único programa. Um arquivo com extensão `.py` pode ser escrito em diferentes editores e ambientes, desde que um interpretador compatível e as bibliotecas necessárias estejam disponíveis.

É útil distinguir três conceitos:

- um **editor de código** oferece recursos para escrever e organizar arquivos;
- uma **IDE**, ou ambiente integrado de desenvolvimento, reúne editor, execução, depuração, terminal e ferramentas de projeto;
- um **notebook** organiza texto, equações, código e resultados em células executáveis.

Entre as alternativas mais conhecidas estão:

Visual Studio Code: editor extensível que, com a extensão Python, oferece execução, depuração, terminal integrado e suporte a notebooks. Será o ambiente principal de validação deste livro.

JupyterLab: ambiente local orientado a notebooks. É especialmente útil para exploração de dados, testes rápidos e combinação de explicações com resultados computacionais.

Google Colab: serviço executado no navegador e baseado em notebooks. Permite escrever e executar Python sem instalar previamente um ambiente completo no computador. A sessão é remota e temporária; arquivos e bibliotecas instalados durante uma sessão podem não permanecer disponíveis na sessão seguinte.

PyCharm: IDE dedicada ao desenvolvimento em Python, com ferramentas de navegação, testes e depuração de projetos.

Spyder: IDE voltada à computação científica, com editor, console interativo e visualização de variáveis.

Também é possível executar Python diretamente em um terminal ou utilizar outros editores. A escolha do ambiente não altera as leis físicas nem a sintaxe fundamental da linguagem, mas pode mudar a forma de executar arquivos, instalar bibliotecas e visualizar gráficos.

Os exemplos deste livro serão escritos como scripts Python portáveis. Sempre que um recurso depender especificamente de notebook, VS Code ou Colab, essa dependência será indicada. Para garantir que os resultados publicados possam ser reproduzidos, a validação oficial será realizada no ambiente virtual do projeto com Python 3.12.

0.4 O primeiro programa

A função `print` envia uma representação de um valor para a saída do programa:

```
1 print("Fisica com Python")
```

O texto entre aspas é uma *string*, isto é, uma sequência de caracteres. Ao executar a instrução, o terminal apresenta

```
Fisica com Python
```

Os parênteses delimitam os argumentos. Aqui, o argumento é uma string.

0.5 Valores, variáveis e atribuição

Uma variável é um nome associado a um valor. A instrução

```
1 massa = 2.5
```

associa o nome `massa` ao valor numérico `2.5`. O símbolo `=` realiza uma **atribuição**; ele não afirma uma igualdade algébrica permanente.

Depois de

```
1 velocidade = 3.0
```

```
2 velocidade = 5.0
```

o nome `velocidade` está associado a `5.0`. A segunda atribuição substitui a referência anterior.

Nomes de variáveis devem comunicar seu significado. Usaremos, por exemplo,

```
1 tempo_total = 12.0
```

```
2 velocidade_inicial = 4.5
```

em vez de nomes genéricos como **a** ou **x1**, exceto quando a notação matemática justificar nomes curtos.

O Python não associa unidades físicas automaticamente a números comuns. Por isso, as unidades devem ser declaradas no texto, nos comentários e nos rótulos das saídas:

```
1 distancia = 120.0 # metros
2 tempo = 10.0      # segundos
```

O caractere **#** inicia um comentário. O interpretador ignora o restante daquela linha, mas o comentário ajuda uma pessoa a compreender o programa.

0.6 Tipos fundamentais

Cada valor possui um tipo. Entre os tipos fundamentais estão:

Tipo	Significado	Exemplo
int	número inteiro	12
float	número de ponto flutuante	9.81
str	sequência de caracteres	tempo
bool	valor lógico	True, False

A função **type** informa o tipo de um valor:

```
1 print(type(12))
2 print(type(9.81))
```

A saída esperada é

```
<class 'int'>
<class 'float'>
```

Um **float** é uma aproximação computacional de um número real. Como sua representação possui precisão finita, algumas operações produzem pequenas diferenças de arredondamento. Esse comportamento será considerado sempre que compararmos resultados numéricos.

0.7 Operadores aritméticos

Os operadores aritméticos básicos são:

Operador	Operação	Exemplo
+	adição	a + b
-	subtração	a - b
*	multiplicação	a * b
/	divisão	a / b
**	potência	a ** 2
//	divisão inteira	a // b
%	resto da divisão	a % b

A precedência das operações é semelhante à utilizada na álgebra. Parênteses devem ser usados quando tornam a intenção mais clara:

```
1 energia = 0.5 * massa * velocidade**2
2 media = (medida_1 + medida_2) / 2
```

0.8 Apresentação de resultados

Uma *f-string* permite inserir valores em um texto:

```
1 energia = 9.0
2 print(f"Energia cinetica: {energia:.2f} J")
```

A especificação `:.2f` apresenta o número com duas casas decimais. A saída será

```
Energia cinetica: 9.00 J
```

Formatar um valor não aumenta a precisão do cálculo. A formatação controla apenas como o número é exibido.

0.9 Funções

Uma função reúne instruções associadas a uma tarefa. Ela pode receber valores de entrada, chamados argumentos, e devolver um resultado:

```
1 def energia_cinetica(massa, velocidade):
2     return 0.5 * massa * velocidade**2
```

A palavra **def** inicia a definição; os dois-pontos abrem um bloco; a indentação identifica as instruções pertencentes à função; e **return** devolve o resultado.

A função pode ser utilizada por meio de uma chamada:

```
1 energia = energia_cinetica(2.0, 3.0)
2 print(energia)
```

A saída esperada é

```
9.0
```

Separar um cálculo em uma função reduz repetição, facilita testes e explicita o modelo empregado.

0.10 Módulos e bibliotecas

Um módulo é um arquivo que contém definições Python reutilizáveis. Uma biblioteca é uma coleção organizada de módulos. O comando **import** torna essas definições disponíveis no programa.

O módulo **math** pertence à biblioteca padrão, instalada juntamente com o Python:

```
1 import math
2
3 raiz = math.sqrt(25)
4 angulo = math.radians(30)
```

O prefixo **math.** informa em qual módulo a função foi definida e evita ambiguidades.

Bibliotecas externas são instaladas separadamente. Neste livro, as principais serão:

- **NumPy**: arrays e computação numérica;
- **Matplotlib**: gráficos e visualizações;
- **SciPy**: algoritmos científicos;
- **SymPy**: matemática simbólica.

Um exemplo de importação é

```
1 import numpy as np
2 import matplotlib.pyplot as plt
```

Os nomes **np** e **plt** são convenções amplamente utilizadas. Eles são aliases: nomes abreviados associados aos módulos importados.

0.11 Arquivos, pastas e caminhos com pathlib

Os programas que produzem gráficos precisam informar onde os arquivos serão guardados. Um caminho completo preso a uma pasta pessoal faria o código funcionar apenas naquela organização. Para criar caminhos portáteis, usaremos a classe **Path**, pertencente ao módulo **pathlib** da biblioteca padrão do Python.

```
1 from pathlib import Path
2
3 pasta_atual = Path.cwd()
4 print(pasta_atual)
```

O método **Path.cwd()** informa a **pasta de trabalho atual**: o local a partir do qual o programa está sendo executado. A saída depende do computador e da pasta aberta no terminal. No Windows, ela poderia ser

```
C:\Users\ana\FisicaComPython\livro
```

No Linux ou no Google Colab, o mesmo comando pode produzir algo como

```
/home/ana/FisicaComPython/livro
```

Essas diferenças não exigem alterações no restante do programa. Em vez de escrever manualmente barras próprias de cada sistema, o operador **/** combina partes de um caminho:

```
1 pasta_figuras = Path("figuras") / "cap00"
2 arquivo = pasta_figuras / "meu-grafico.pdf"
```

Como o caminho começa com **figuras** e não com uma unidade de disco ou uma barra inicial, ele é **relativo**. Se a pasta de trabalho for **livro**, o arquivo ficará em **livro/figuras/cap00/meu-grafico.pdf**. O método **resolve** permite visualizar o caminho completo calculado no computador:

```
1 print(arquivo.resolve())
```

Antes de salvar, precisamos garantir que a pasta exista:

```
1 pasta_figuras.mkdir(parents=True, exist_ok=True)
```

O argumento **parents=True** autoriza a criação de pastas intermediárias; **exist_ok=True** evita um erro quando a pasta já existe. A sequência portátil que adotaremos será, portanto,

```
1 from pathlib import Path
2
```

```
3 pasta_figuras = Path("figuras") / "cap00"  
4 pasta_figuras.mkdir(parents=True, exist_ok=True)  
5 arquivo = pasta_figuras / "meu-grafico.pdf"
```

Nos códigos oficiais deste livro, o terminal será aberto na pasta **livro**. Por exemplo,

```
python codigos/cap00/primeiro_grafico.py
```

Assim, os programas guardam os resultados na pasta **figuras**, dentro de **livro**.

Se o leitor copiar um script para outro projeto, a mesma instrução criará a pasta **figuras** no novo local de execução.

Também existe a variável especial `__file__`, que informa o arquivo do script. Ela pode ser útil em projetos com estruturas rígidas, mas não está disponível da mesma forma em todos os notebooks. Por isso, os exemplos introdutórios usarão caminhos relativos à pasta de trabalho. No VS Code, no JupyterLab e no Colab, o leitor deve conferir `Path.cwd()` quando tiver dúvida sobre o destino de um arquivo.

0.12 Primeiros gráficos com Matplotlib

Um gráfico científico não é apenas uma ilustração: ele representa a relação entre grandezas. Antes de interpretar curvas de posição, velocidade, energia ou campo, precisamos compreender como os dados são convertidos em elementos visuais e como esses elementos são construídos em Python.

Usaremos o módulo `matplotlib.pyplot`, importado com o alias `plt`. A biblioteca Matplotlib organiza um gráfico em dois objetos principais:

Figura (fig): a área completa que será exibida ou salva, como uma folha que pode conter um ou mais gráficos;

Eixos (ax): a região em que os dados são desenhados, incluindo os eixos coordenados, marcas, rótulos e curvas.

Essa distinção explica a instrução

```
1 fig, ax = plt.subplots()
```

A função `plt.subplots` cria a figura e um sistema de eixos. Os nomes **fig** e **ax** são convenções, não palavras reservadas do Python.

0.12.1 Um primeiro gráfico a partir de dados

Considere posições medidas a cada segundo. Duas listas armazenam os valores correspondentes de tempo e posição. O primeiro elemento de **tempo** é associado ao primeiro elemento de **posicao**, e assim sucessivamente.

Código 1: Primeiro gráfico construído a partir de dados de posição e tempo.

```
1  """Primeiro gráfico: posição medida em diferentes instantes."""
2
3  from pathlib import Path
4
5  import matplotlib.pyplot as plt
6
7  tempo = [0, 1, 2, 3, 4]
8  posicao = [0, 2, 4, 6, 8]
9
10 fig, ax = plt.subplots(figsize=(7, 4))
11 ax.plot(tempo, posicao, marker="o")
12 ax.set_xlabel("Tempo (s)")
13 ax.set_ylabel("Posição (m)")
14 ax.set_title("Movimento uniforme")
15 fig.tight_layout()
16
17 pasta_figuras = Path("figuras") / "cap00"
18 pasta_figuras.mkdir(parents=True, exist_ok=True)
19 arquivo = pasta_figuras / "primeiro-grafico.pdf"
20 fig.savefig(arquivo, bbox_inches="tight")
21 plt.close(fig)
22
23 print("Gráfico salvo em figuras/cap00/primeiro-grafico.pdf")
```

A saída textual esperada é

Código 2: Saída esperada do primeiro programa gráfico.

```
Gráfico salvo em figuras/cap00/primeiro-grafico.pdf
```

O arquivo produzido pelo programa é mostrado na [figura 1](#).

No [código 1](#):

- `ax.plot(tempo, posicao)` associa os valores da primeira sequência ao eixo horizontal e os da segunda ao eixo vertical;
- o argumento `marker="o"` marca cada par de dados com um círculo;
- `set_xlabel` e `set_ylabel` identificam as grandezas e suas unidades;

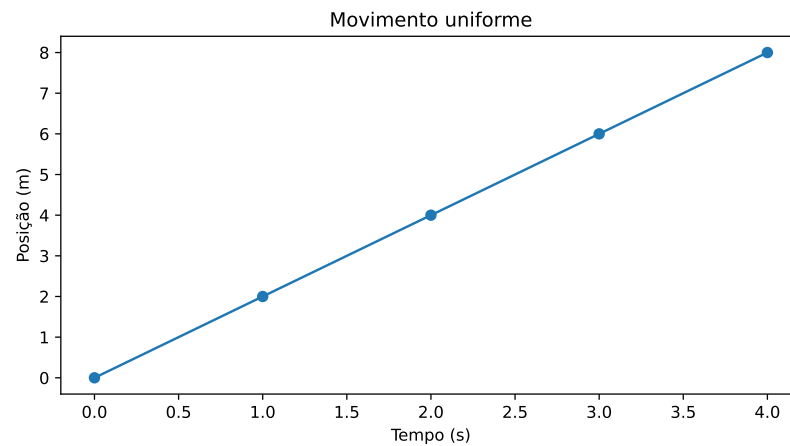


Figura 1: Posição em função do tempo. Os círculos identificam os dados fornecidos ao programa e os segmentos apenas orientam a leitura entre eles.

- `set_title` acrescenta um título descritivo;
- `tight_layout` ajusta os espaços para evitar cortes nos rótulos;
- `savefig` salva a figura, enquanto `plt.close` libera a memória usada por ela.

Durante uma exploração interativa, `plt.show()` pode ser usado para abrir a figura em uma janela ou exibi-la em um notebook. Nos scripts do livro, preferimos `savefig`: o resultado fica registrado, pode ser reproduzido e pode ser inserido no texto. O formato PDF preserva linhas e letras como elementos vetoriais, adequados à impressão.

0.12.2 Amostragem de um intervalo com NumPy

Listas são suficientes para armazenar uma pequena quantidade de dados experimentais. Para representar uma curva, calculamos a função em muitos pontos.

Por exemplo,

```
1 tempo = np.linspace(0, 5, 101)
```

cria um array NumPy com 101 valores igualmente espaçados, incluindo 0 e 5. Um *array* é uma estrutura numérica que permite aplicar uma operação a todos os seus elementos. Assim,

```
1 posicao = 2.0 * tempo
```

calcula uma posição para cada instante sem escrever um laço explicitamente. O conjunto de valores escolhidos para a variável independente é o **domínio amostrado**. A curva exibida é formada por segmentos entre os pontos calculados; ela não contém infinitos pontos. Aumentar a amostragem pode tornar a curva mais suave, mas não corrige um modelo físico inadequado.

0.12.3 Eixos, estilos, legenda, grade e anotação

Quando duas ou mais curvas aparecem nos mesmos eixos, cada uma precisa ser identificável também em impressão sem cores. Para isso, combinamos rótulos com estilos de linha diferentes.

Código 3: Elementos essenciais de um gráfico científico.

```
1  """Elementos de um gráfico científico com duas séries de dados."""
2
3  from pathlib import Path
4
5  import matplotlib.pyplot as plt
6  import numpy as np
7
8  tempo = np.linspace(0, 5, 101)
9  posicao_a = 2.0 * tempo
10 posicao_b = 1.0 + 1.2 * tempo
11
12 fig, ax = plt.subplots(figsize=(7, 4.5))
13 ax.plot(tempo, posicao_a, color="tab:blue", label="Objeto A")
14 ax.plot(
15     tempo,
16     posicao_b,
17     color="tab:orange",
18     linestyle="--",
19     label="Objeto B",
20 )
21 ax.set_xlabel("Tempo (s)")
22 ax.set_ylabel("Posição (m)")
23 ax.set_title("Comparação entre dois movimentos")
24 ax.set_xlim(0, 5)
25 ax.set_ylim(0, 11)
26 ax.grid(alpha=0.3)
27 ax.legend()
28 ax.annotate(
29     "mesma posição",
```

```
30     xy=(1.25, 2.5),
31     xytext=(2.0, 1.2),
32     arrowprops={"arrowstyle": "->"},
33 )
34 fig.tight_layout()
35
36 pasta_figuras = Path("figuras") / "cap00"
37 pasta_figuras.mkdir(parents=True, exist_ok=True)
38 arquivo = pasta_figuras / "elementos-grafico.pdf"
39 fig.savefig(arquivo, bbox_inches="tight")
40 plt.close(fig)
41
42 print(f"Foram calculados {tempo.size} pontos por curva.")
43 print("Gráfico salvo em figuras/cap00/elementos-grafico.pdf")
```

A saída esperada é

Código 4: Saída esperada do programa com duas curvas.

```
Foram calculados 101 pontos por curva.
Gráfico salvo em figuras/cap00/elementos-grafico.pdf
```

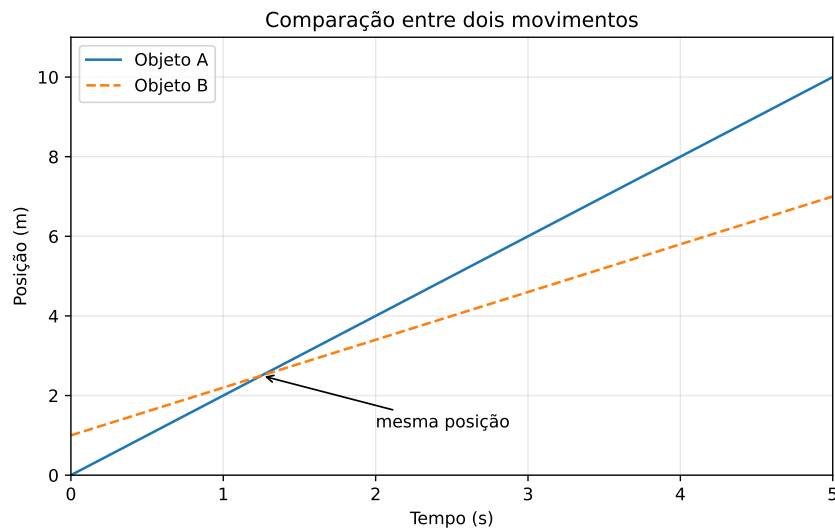


Figura 2: Dois movimentos apresentados nos mesmos eixos. Cor, estilo de linha, legenda e anotação trabalham juntos para comunicar a informação.

Os novos comandos controlam diferentes partes da comunicação visual:

color: escolhe a cor da curva;

linestyle: define o estilo, como linha contínua ou tracejada;

label: associa um texto à curva;

legend: exibe os textos associados às curvas;

set_xlim e set_ylim: definem os limites visíveis dos eixos;

grid: acrescenta linhas de referência; o parâmetro **alpha** controla sua transparência;

annotate: relaciona uma observação a um ponto do gráfico. O argumento **xy** indica o ponto e **xytext** indica a posição do texto.

Limites de eixos devem ser escolhidos com critério. Cortar parte da escala pode destacar uma variação pequena, mas também pode exagerar visualmente diferenças. Uma grade deve ajudar na leitura, não competir com os dados. Uma legenda é necessária quando há várias curvas sem identificação direta; em uma curva única, bons rótulos de eixos e uma legenda de figura geralmente bastam.

0.12.4 Mais de um sistema de eixos

A função `plt.subplots` também cria vários gráficos na mesma figura:

```
1 fig, eixos = plt.subplots(1, 2)
2 eixos[0].plot(x, y_1)
3 eixos[1].plot(x, y_2)
```

Os argumentos `1, 2` especificam uma linha e duas colunas. Nesse caso, `eixos` é uma coleção: `eixos[0]` seleciona o primeiro sistema de eixos e `eixos[1]`, o segundo. Esse recurso será usado quando a comparação lado a lado for mais clara do que sobrepor muitas curvas.

0.12.5 Linhas de referência, marcas, proporção e formas

Alguns gráficos exigem mais do que curvas. Os comandos a seguir serão úteis na trigonometria e na representação de vetores:

```
1 ax.axhline(0)                # linha horizontal em y = 0
2 ax.axvline(0)                # linha vertical em x = 0
3 ax.set_xticks([0, 1, 2])     # posições das marcas
4 ax.set_aspect("equal")       # mesma escala visual nos eixos
5 circulo = plt.Circle((0, 0), 1, fill=False)
```

```
6 ax.add_patch(circulo)          # adiciona a forma aos eixos
7 ax.quiver(0, 0, 1, 1)         # seta de (0, 0) até (1, 1)
```

Os métodos **axhline** e **axvline** criam linhas de referência paralelas aos eixos. O método **set_xticks** controla onde aparecem as marcas do eixo horizontal; um segundo argumento opcional pode fornecer textos personalizados, como $\pi/2$ e π . O ajuste **set_aspect("equal")** faz uma unidade horizontal ocupar o mesmo comprimento visual que uma unidade vertical, condição necessária para que um círculo não pareça uma elipse.

Uma **Circle** é uma forma geométrica que só passa a pertencer ao gráfico depois de ser entregue a **add_patch**. O método **quiver** desenha setas e será estudado com mais profundidade quando representarmos vetores. Por enquanto, basta observar que os dois primeiros números indicam a origem e os dois últimos, as componentes da seta.

Textos matemáticos podem ser escritos entre sinais de dólar. Usaremos frequentemente strings como **r"\$\sin(\theta)\$"**. O prefixo **r** cria uma *raw string*, na qual as barras invertidas são preservadas para a notação matemática do Matplotlib.

0.12.6 Critérios para um gráfico científico

Nem todo cálculo precisa de um gráfico. Ele é útil quando a forma, a tendência, a comparação ou a variação entre grandezas acrescenta informação. Quando for usado, verificaremos se:

1. os eixos identificam grandezas e unidades;
2. o domínio e os limites correspondem ao problema físico;
3. curvas diferentes podem ser distinguidas sem depender apenas de cor;
4. a legenda existe somente quando necessária;
5. a quantidade de pontos representa adequadamente a função ou os dados;
6. título, grade e anotações ajudam a leitura sem criar excesso visual;
7. o arquivo foi salvo com resolução ou formato apropriado.

0.13 Ambiente virtual e dependências

Um ambiente virtual isola o interpretador e as bibliotecas de um projeto. Assim, projetos diferentes podem utilizar versões diferentes de uma mesma biblioteca sem interferência entre si.

Neste repositório, o ambiente local fica na pasta `.venv`. No terminal do VS Code para Windows, ele pode ser ativado com

```
.\.venv\Scripts\Activate.ps1
```

As dependências declaradas no arquivo `requirements.txt` podem ser instaladas com

```
python -m pip install -r requirements.txt
```

O comando `python -m pip` garante que o instalador utilizado pertence ao interpretador Python selecionado.

0.14 Erros como informação

Uma mensagem de erro não significa apenas que o programa falhou. Ela registra onde ocorreu o problema e qual regra foi violada. Três categorias aparecerão com frequência:

- **SyntaxError**: a instrução não segue a sintaxe da linguagem;
- **NameError**: um nome foi usado sem estar definido;
- **TypeError**: uma operação recebeu um tipo incompatível.

Ao investigar um erro, devemos ler a última linha da mensagem e localizar o arquivo e a linha indicados. Corrigir erros faz parte do processo normal de modelagem computacional.

0.15 Critério de validação

Antes de considerar um código aprovado, verificaremos:

1. se o programa executa sem erros;
2. se as unidades utilizadas são consistentes;
3. se o resultado coincide com um caso calculado manualmente;

4. se casos-limite produzem comportamento coerente;
5. se nomes, comentários e saídas são compreensíveis;
6. se gráficos possuem eixos, unidades e legendas adequados.

Esses critérios acompanharão todos os códigos do livro.

Capítulo 1

Ferramentas matemáticas para a Física

1.1 Por que começar pela matemática?

A Física procura descrever regularidades da natureza. Para isso, transforma observações em grandezas mensuráveis e relações entre essas grandezas. A matemática não substitui o fenômeno: ela oferece uma linguagem compacta para formular um modelo, extrair consequências e comparar previsões com medidas.

Considere uma pessoa que caminha em linha reta. A frase “ela se afasta da origem mantendo o mesmo ritmo” pode ser representada por

$$x(t) = x_0 + vt, \quad (1.1)$$

em que x é a posição, t é o instante, x_0 é a posição inicial e v é a velocidade constante. A equação não é o movimento real; é um modelo que conserva apenas as características relevantes para a pergunta investigada.

Neste capítulo, revisaremos as ferramentas matemáticas que aparecerão repetidamente no livro. O objetivo não é apresentar um curso completo de matemática, mas construir um vocabulário operacional para interpretar equações, gráficos e variações.

1.2 Potências e raízes

Uma potência representa uma multiplicação repetida. Para um expoente inteiro positivo n ,

$$a^n = \underbrace{a \cdot a \cdot a \cdots a}_{n \text{ fatores}}. \quad (1.2)$$

Por exemplo,

$$3^4 = 3 \cdot 3 \cdot 3 \cdot 3 = 81. \quad (1.3)$$

Para uma mesma base não nula, valem as propriedades

$$a^m a^n = a^{m+n}, \quad (1.4)$$

$$\frac{a^m}{a^n} = a^{m-n}, \quad (1.5)$$

$$(a^m)^n = a^{mn}, \quad (1.6)$$

$$a^{-n} = \frac{1}{a^n}, \quad (1.7)$$

$$a^0 = 1. \quad (1.8)$$

Potências fracionárias representam raízes. Para $a \geq 0$,

$$a^{1/2} = \sqrt{a} \quad (1.9)$$

e

$$a^{1/n} = \sqrt[n]{a}. \quad (1.10)$$

Essas propriedades aparecem frequentemente na manipulação de fórmulas físicas. Se uma grandeza depende de x^2 , por exemplo, multiplicar x por 3 multiplica essa grandeza por $3^2 = 9$.

1.2.1 Potências em Python

Em Python, o operador de potência é formado por dois asteriscos, ******. O símbolo não representa potência: ele executa uma operação binária chamada **XOR**.

Código 1.1: Potências, raízes e uma aplicação à energia cinética.

```
1  """Potencias, raizes e uma verificacao de lei de escala."""
2
3  quadrado = 5**2
4  cubo = 3**3
5  potencia_negativa = 10**-3
```

```
6 raiz_quadrada = 25**0.5
7 raiz_cubica = 64 ** (1 / 3)
8
9 print(f"5 ao quadrado = {quadrado}")
10 print(f"3 ao cubo = {cubo}")
11 print(f"10 elevado a -3 = {potencia_negativa}")
12 print(f"Raiz quadrada de 25 = {raiz_quadrada}")
13 print(f"Raiz cubica de 64 = {raiz_cubica}")
14 print(f"Raiz cubica de 64, formatada = {raiz_cubica:.2f}")
15
16 massa = 2.0 # kg
17 velocidade_1 = 3.0 # m/s
18 velocidade_2 = 3 * velocidade_1
19
20 energia_1 = 0.5 * massa * velocidade_1**2
21 energia_2 = 0.5 * massa * velocidade_2**2
22
23 fator_velocidade = velocidade_2 / velocidade_1
24 fator_energia = energia_2 / energia_1
25
26 print(f"Energia cinetica inicial = {energia_1:.2f} J")
27 print(f"Energia cinetica final = {energia_2:.2f} J")
28 print(f"Fator da velocidade = {fator_velocidade:.0f}")
29 print(f"Fator da energia = {fator_energia:.0f}")
```

Código 1.2: Saída esperada do programa de potências e escalas.

```
5 ao quadrado = 25
3 ao cubo = 27
10 elevado a -3 = 0.001
Raiz quadrada de 25 = 5.0
Raiz cubica de 64 = 3.9999999999999996
Raiz cubica de 64, formatada = 4.00
Energia cinetica inicial = 9.00 J
Energia cinetica final = 81.00 J
Fator da velocidade = 3
Fator da energia = 9
```

O valor calculado para $64^{1/3}$ pode aparecer como 3,9999999999999996 em vez de 4. Isso ocorre porque muitos números reais não possuem representação binária finita no computador. A formatação `:.2f` controla a apresentação do resultado, mas não altera o valor armazenado.

No trecho final do [código 1.1](#), triplicamos a velocidade e verificamos numericamente que a energia cinética é multiplicada por nove. O programa,

portanto, não apenas calcula um valor: ele testa uma previsão obtida pela análise matemática.

1.3 Notação científica e ordem de grandeza

A Física estuda fenômenos em escalas muito diferentes. O diâmetro aproximado de um próton é da ordem de 10^{-15} m, enquanto a distância entre estrelas pode chegar a muitos trilhões de quilômetros. Escrever números tão pequenos ou tão grandes na forma decimal dificulta cálculos e comparações.

A notação científica representa um número na forma

$$N = a \times 10^n, \quad 1 \leq |a| < 10, \quad (1.11)$$

em que n é um número inteiro.

Por exemplo,

$$0,000\,004\,8\text{ s} = 4,8 \cdot 10^{-6}\text{ s}. \quad (1.12)$$

O expoente -6 informa que a vírgula foi deslocada seis posições para a direita para obter o número 4,8.

Para um número grande,

$$384\,000\,000\text{ m} = 3,84 \cdot 10^8\text{ m}. \quad (1.13)$$

Nesse caso, a vírgula foi deslocada oito posições para a esquerda.

1.3.1 Operações com notação científica

Na multiplicação, multiplicamos os coeficientes e somamos os expoentes:

$$(a \times 10^m)(b \times 10^n) = ab \times 10^{m+n}. \quad (1.14)$$

Como exemplo,

$$(3 \times 10^4)(2 \times 10^{-2}) = 6 \times 10^2. \quad (1.15)$$

Na divisão, dividimos os coeficientes e subtraímos os expoentes:

$$\frac{a \times 10^m}{b \times 10^n} = \frac{a}{b} \times 10^{m-n}. \quad (1.16)$$

Portanto,

$$\frac{8 \times 10^7}{2 \times 10^3} = 4 \times 10^4. \quad (1.17)$$

Na adição e na subtração, os expoentes precisam ser iguais. Por exemplo,

$$3,2 \times 10^5 + 7,0 \times 10^4 = 3,2 \times 10^5 + 0,7 \times 10^5 \quad (1.18)$$

$$= 3,9 \times 10^5. \quad (1.19)$$

1.3.2 Notação científica em Python

Python aceita a letra **e** para representar potências de 10. Assim, **4.8e-6** corresponde a $4,8 \times 10^{-6}$. A especificação de formato **.2e** apresenta um número em notação científica com duas casas decimais.

Código 1.3: Representação e operações com notação científica.

```
1  """Representacao de numeros em notacao scientifica."""
2
3  tempo = 4.8e-6 # s
4  distancia = 3.84e8 # m
5
6  print(f"Tempo na forma decimal: {tempo}")
7  print(f"Tempo em notacao scientifica: {tempo:.2e} s")
8  print(f"Distancia em notacao scientifica: {distancia:.2e} m")
9
10 produto = (3e4) * (2e-2)
11 quociente = (8e7) / (2e3)
12
13 print(f"Produto: {produto:.2e}")
14 print(f"Quociente: {quociente:.2e}")
```

Código 1.4: Saída esperada do programa de notação científica.

```
Tempo na forma decimal: 4.8e-06
Tempo em notacao scientifica: 4.80e-06 s
Distancia em notacao scientifica: 3.84e+08 m
Produto: 6.00e+02
Quociente: 4.00e+04
```

1.3.3 Ordem de grandeza

A ordem de grandeza é a potência de dez mais próxima do valor considerado. Ela permite comparar escalas e avaliar rapidamente se um resultado é

plausível.

Considere

$$N = a \times 10^n, \quad 1 \leq a < 10. \quad (1.20)$$

Como o ponto médio entre 10^n e 10^{n+1} , em escala logarítmica, corresponde a $\sqrt{10} \approx 3,16$, adotamos

$$\text{OG}(N) = \begin{cases} 10^n, & a < \sqrt{10}, \\ 10^{n+1}, & a \geq \sqrt{10}. \end{cases} \quad (1.21)$$

Assim, $2,4 \times 10^6$ tem ordem de grandeza 10^6 , enquanto $7,1 \times 10^6$ tem ordem de grandeza 10^7 .

1.3.4 Estimativas físicas

Antes de aceitar o resultado de um cálculo, devemos perguntar:

- A unidade está correta?
- O sinal possui significado físico?
- O valor está na escala esperada?
- O resultado respeita os limites do modelo?

Uma velocidade cotidiana de 3×10^7 m/s é matematicamente possível como número, mas incompatível com uma caminhada, um automóvel ou um avião. Um resultado assim provavelmente indica erro de unidade, entrada ou interpretação.

Exemplo resolvido

Um sinal percorre $1,5 \times 10^8$ m em $5,0 \times 10^{-1}$ s. Sua velocidade média é

$$v_m = \frac{\Delta s}{\Delta t}. \quad (1.22)$$

Substituindo,

$$v_m = \frac{1,5 \times 10^8}{5,0 \times 10^{-1}} \quad (1.23)$$

$$= \frac{1,5}{5,0} \times 10^{8-(-1)} \quad (1.24)$$

$$= 0,30 \times 10^9 \quad (1.25)$$

$$= 3,0 \times 10^8 \text{ m/s.} \quad (1.26)$$

Além de efetuar o cálculo, devemos interpretar o resultado. O valor está na ordem de grandeza da velocidade da luz no vácuo, indicando que o modelo pode estar descrevendo a propagação de um sinal eletromagnético.

Verificação conceitual

1. Escreva 0,000000032 m em notação científica.
2. Escreva $6,02 \times 10^{23}$ na forma decimal.
3. Determine a ordem de grandeza de $2,8 \times 10^5$.
4. Determine a ordem de grandeza de $8,4 \times 10^5$.
5. Explique por que a ordem de grandeza é útil para validar resultados físicos.

1.4 Proporcionalidade e leis de escala

As relações de proporcionalidade ajudam a prever como uma grandeza física muda quando outra é alterada. Muitas vezes, essa análise pode ser feita antes de qualquer substituição numérica.

1.4.1 Proporcionalidade direta

Quando escrevemos

$$y \propto x, \quad (1.27)$$

afirmamos que existe uma constante k tal que

$$y = kx. \quad (1.28)$$

A razão entre as duas grandezas permanece constante:

$$\frac{y}{x} = k. \quad (1.29)$$

Se multiplicarmos x por um fator f , o mesmo acontecerá com y :

$$x' = fx \Rightarrow y' = fy. \quad (1.30)$$

Por exemplo, se a distância percorrida por um objeto com velocidade constante é

$$d = vt, \quad (1.31)$$

então, mantendo v constante,

$$d \propto t. \quad (1.32)$$

Se o intervalo de tempo dobrar, a distância também dobrará.

1.4.2 Proporcionalidade com uma potência

Uma relação mais geral pode ser escrita como

$$y \propto x^n, \quad (1.33)$$

ou

$$y = kx^n. \quad (1.34)$$

Quando x é multiplicado por um fator f ,

$$x' = fx, \quad (1.35)$$

temos

$$y' = k(fx)^n = f^n kx^n. \quad (1.36)$$

Portanto,

$$\frac{y'}{y} = f^n. \quad (1.37)$$

Essa expressão permite prever a mudança de y sem conhecer o valor da constante k .

Exemplo resolvido: energia cinética

A energia cinética de um corpo de massa m e velocidade v é

$$E_c = \frac{1}{2}mv^2. \quad (1.38)$$

Mantendo a massa constante,

$$E_c \propto v^2. \quad (1.39)$$

Se a velocidade for triplicada, $v' = 3v$. Assim,

$$\frac{E'_c}{E_c} = \left(\frac{v'}{v}\right)^2 = 3^2 = 9. \quad (1.40)$$

Portanto, triplicar a velocidade multiplica a energia cinética por nove. A dependência quadrática faz a energia crescer mais rapidamente que a velocidade.

1.4.3 Proporcionalidade inversa

Quando

$$y \propto \frac{1}{x}, \quad (1.41)$$

temos

$$y = \frac{k}{x}. \quad (1.42)$$

Nesse caso, $xy = k$. Se x for duplicado, y será reduzido à metade. De maneira mais geral,

$$y \propto \frac{1}{x^n}. \quad (1.43)$$

Se x for multiplicado por f , então

$$\frac{y'}{y} = \frac{1}{f^n}. \quad (1.44)$$

Exemplo resolvido: intensidade e distância

Considere um modelo idealizado no qual a intensidade I recebida de uma fonte puntiforme é inversamente proporcional ao quadrado da distância r :

$$I \propto \frac{1}{r^2}. \quad (1.45)$$

Queremos comparar a intensidade a uma distância r com a intensidade a uma distância $4r$:

$$\frac{I(4r)}{I(r)} = \frac{1/(4r)^2}{1/r^2} = \frac{1}{16}. \quad (1.46)$$

Ao quadruplicar a distância, a intensidade é reduzida a um dezesseis avos do valor inicial. Esse comportamento pode ser compreendido geometricamente: a mesma quantidade emitida pela fonte distribui-se por uma área que cresce proporcionalmente a r^2 .

1.4.4 Proporcionalidade conjunta

Uma grandeza pode depender de mais de uma variável. Por exemplo,

$$y \propto \frac{a^2 b}{c^3}. \quad (1.47)$$

Nesse caso,

$$y = k \frac{a^2 b}{c^3}. \quad (1.48)$$

Se a dobrar, b for triplicado e c permanecer constante, então

$$\frac{y'}{y} = 2^2 \cdot 3 = 12. \quad (1.49)$$

A grandeza y será multiplicada por 12.

1.4.5 Isolamento de variáveis

As fórmulas físicas também precisam ser reorganizadas de acordo com a grandeza procurada. Considere

$$E_c = \frac{1}{2}mv^2. \quad (1.50)$$

Para isolar v , multiplicamos a equação por 2:

$$2E_c = mv^2. \quad (1.51)$$

Dividimos por m :

$$v^2 = \frac{2E_c}{m}. \quad (1.52)$$

Finalmente,

$$v = \sqrt{\frac{2E_c}{m}}. \quad (1.53)$$

Como v representa o módulo da velocidade, escolhemos a raiz não negativa. Em outros contextos, uma equação quadrática pode admitir soluções positivas e negativas com significados físicos diferentes.

1.4.6 Limites de uma proporcionalidade

Uma proporcionalidade física geralmente vale dentro de determinadas condições. A expressão

$$F = k\Delta x \quad (1.54)$$

descreve aproximadamente uma mola enquanto ela permanece em seu regime elástico. Um alongamento excessivo pode deformar permanentemente o material, tornando o modelo inadequado.

Portanto, identificar uma proporcionalidade exige também perguntar:

- Quais grandezas permanecem constantes?
- Em qual intervalo a relação foi observada?
- Quais hipóteses foram utilizadas?
- Em que situação o modelo deixa de funcionar?

Verificação conceitual

1. Se $y \propto x^3$, por qual fator y muda quando x dobra?
2. Se $y \propto 1/x^2$, por qual fator y muda quando x é dividido por 3?
3. Com velocidade constante, o que ocorre com E_c se a massa quintuplica?

4. Com massa constante, o que ocorre com v para E_c quadruplicar?
5. Considere $P \propto A^2/(BC)$. Determine a mudança em P quando A dobra, B triplica e C é reduzido à metade.
6. Explique por que uma proporcionalidade correta pode deixar de representar um sistema físico real.

1.5 Logaritmos e escalas logarítmicas

O logaritmo responde à seguinte pergunta: a qual expoente devemos elevar uma base para obter determinado número? Se

$$b^y = x, \quad (1.55)$$

então

$$y = \log_b x, \quad (1.56)$$

com as condições $b > 0$, $b \neq 1$ e $x > 0$.

Por exemplo, $10^3 = 1000$ implica

$$\log_{10}(1000) = 3. \quad (1.57)$$

Da mesma forma, $2^{-3} = 1/8$ implica

$$\log_2 \left(\frac{1}{8} \right) = -3. \quad (1.58)$$

1.5.1 Bases mais utilizadas

O logaritmo decimal possui base 10:

$$\log x = \log_{10} x. \quad (1.59)$$

O logaritmo natural possui base $e \approx 2,71828$ e é representado por

$$\ln x = \log_e x. \quad (1.60)$$

O logaritmo natural aparece naturalmente em processos exponenciais, equações diferenciais, oscilações amortecidas e fenômenos de crescimento ou decaimento.

1.5.2 Propriedades fundamentais

Para $a > 0$, $b > 0$ e qualquer número real n ,

$$\log(ab) = \log a + \log b, \quad (1.61)$$

$$\log\left(\frac{a}{b}\right) = \log a - \log b, \quad (1.62)$$

$$\log(a^n) = n \log a. \quad (1.63)$$

Também temos $\log_b 1 = 0$ e $\log_b b = 1$. Essas propriedades transformam multiplicações em adições, divisões em subtrações e potências em produtos.

1.5.3 Mudança de base

Um logaritmo pode ser convertido de uma base para outra por meio de

$$\log_b x = \frac{\log_c x}{\log_c b}. \quad (1.64)$$

Em particular,

$$\log_b x = \frac{\ln x}{\ln b}. \quad (1.65)$$

1.5.4 Logaritmos e leis de potência

Considere uma relação física da forma

$$y = Cx^n, \quad (1.66)$$

em que C e n são constantes. Aplicando o logaritmo aos dois lados,

$$\log y = \log C + n \log x. \quad (1.67)$$

Definindo $Y = \log y$ e $X = \log x$, obtemos

$$Y = nX + \log C. \quad (1.68)$$

Essa é a equação de uma reta. Em um gráfico de $\log y$ em função de $\log x$, a inclinação corresponde ao expoente n , e a interseção com o eixo vertical corresponde a $\log C$.

Exemplo resolvido: determinação de uma lei de escala

Uma grandeza y apresenta os valores

x	y
2	12
4	48

Suponha que $y = Cx^n$. Dividindo as duas equações,

$$\frac{48}{12} = \frac{C4^n}{C2^n}. \quad (1.69)$$

Logo, $4 = 2^n$ e, portanto, $n = 2$. Usando $x = 2$ e $y = 12$,

$$12 = C(2)^2, \quad (1.70)$$

de onde $C = 3$. Assim, a relação encontrada é

$$y = 3x^2. \quad (1.71)$$

1.5.5 Escalas logarítmicas

Uma escala linear representa diferenças iguais por distâncias iguais. Uma escala logarítmica representa razões iguais por distâncias iguais. Em uma escala logarítmica de base 10, os valores 1, 10, 100 e 1000 ficam igualmente espaçados porque cada valor é dez vezes maior que o anterior.

Esse tipo de escala é útil quando uma grandeza percorre muitas ordens de magnitude. Ela permite representar valores muito diferentes no mesmo gráfico sem comprimir excessivamente os menores.

Exemplo físico: nível de intensidade sonora

O nível de intensidade sonora é definido por

$$\beta = 10 \log_{10} \left(\frac{I}{I_0} \right), \quad (1.72)$$

em que β é medido em decibéis, I é a intensidade sonora e I_0 é uma intensidade de referência.

Se $I = 1000I_0$, então

$$\beta = 10 \log_{10}(1000) = 30 \text{ dB}. \quad (1.73)$$

Se a intensidade for multiplicada por 10, o nível aumenta em 10 dB:

$$\beta' = 10 \log_{10} \left(\frac{10I}{I_0} \right) \quad (1.74)$$

$$= 10 \left[\log_{10}(10) + \log_{10} \left(\frac{I}{I_0} \right) \right] \quad (1.75)$$

$$= 10 \text{ dB} + \beta. \quad (1.76)$$

1.5.6 Argumentos dimensionais

O argumento de um logaritmo deve ser adimensional. Por isso, expressões físicas logarítmicas usam razões entre grandezas de mesma dimensão, como

$$\ln \left(\frac{x}{x_0} \right) \quad (1.77)$$

e

$$\log_{10} \left(\frac{I}{I_0} \right). \quad (1.78)$$

Uma expressão como $\ln(3 \text{ m})$ não é fisicamente adequada, pois seu argumento ainda possui dimensão de comprimento.

1.5.7 Logaritmos e exponenciais como operações inversas

As funções exponencial e logarítmica desfazem uma à outra:

$$\ln(e^x) = x \quad (1.79)$$

e, para $x > 0$,

$$e^{\ln x} = x. \quad (1.80)$$

Exemplo resolvido: tempo de decaimento

Considere

$$N(t) = N_0 e^{-t/\tau}. \quad (1.81)$$

Queremos determinar o instante em que $N(t) = N_0/4$. Substituindo e dividindo por N_0 ,

$$\frac{1}{4} = e^{-t/\tau}. \quad (1.82)$$

Aplicando o logaritmo natural,

$$\ln\left(\frac{1}{4}\right) = -\frac{t}{\tau}. \quad (1.83)$$

Como $\ln(1/4) = -\ln 4$, obtemos

$$t = \tau \ln 4 \approx 1,386\tau. \quad (1.84)$$

1.5.8 Logaritmos em Python

O módulo `math` oferece `log10` para o logaritmo decimal e `log` para o logaritmo natural. No mesmo programa, usamos escalas logarítmicas nos dois eixos para visualizar a lei $y = 3x^2$.

Código 1.5: Logaritmos, decaimento exponencial e gráfico log-log.

```

1  """Logaritmos, decaimento exponencial e escalas logaritmicas."""
2
3  from pathlib import Path
4  import math
5
6  import matplotlib.pyplot as plt
7  import numpy as np
8
9  print(f"log10(1000) = {math.log10(1000):.1f}")
10 print(f"ln(4) = {math.log(4):.6f}")
11
12 tau = 2.0 # s
13 tempo_quarto = tau * math.log(4)
14 print(f"Tempo para N0/4: {tempo_quarto:.3f} s")
15
16 x = np.logspace(0, 2, 200)
17 y = 3 * x**2
18
19 fig, ax = plt.subplots(figsize=(7, 4.5))
20 ax.loglog(x, y, color="tab:blue", linewidth=2, label=r"$y=3x^2$")
21 ax.set_xlabel(r"$x$ (escala logaritmica)")
22 ax.set_ylabel(r"$y$ (escala logaritmica)")
23 ax.grid(True, which="both", alpha=0.3)
24 ax.legend()

```

```
25 fig.tight_layout()
26
27 pasta_figuras = Path("figuras") / "cap01"
28 pasta_figuras.mkdir(parents=True, exist_ok=True)
29 arquivo = pasta_figuras / "lei-potencia-loglog.pdf"
30 fig.savefig(arquivo, bbox_inches="tight")
31 plt.close(fig)
32
33 print("Gráfico salvo em figuras/cap01/lei-potencia-loglog.pdf")
```

Código 1.6: Saída esperada do programa de logaritmos.

```
log10(1000) = 3.0
ln(4) = 1.386294
Tempo para N0/4: 2.773 s
Gráfico salvo em figuras/cap01/lei-potencia-loglog.pdf
```

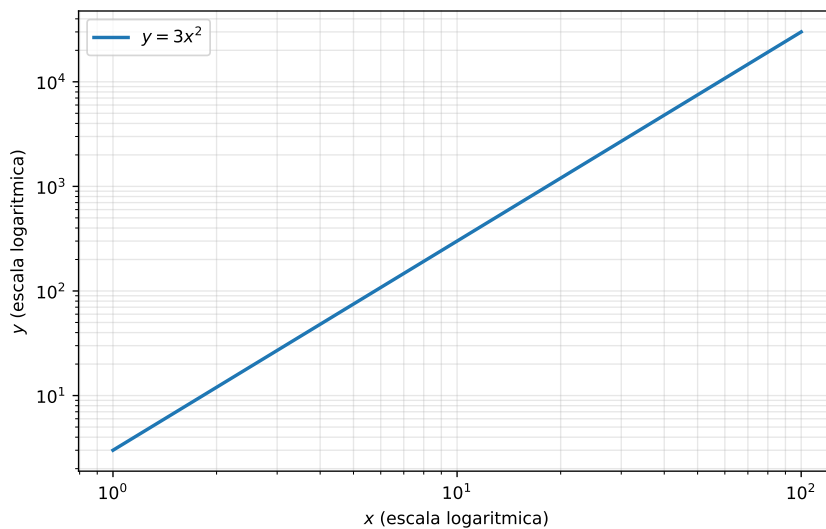


Figura 1.1: A lei de potência $y = 3x^2$ em escalas logarítmicas. A inclinação constante corresponde ao expoente 2.

Verificação conceitual

1. Calcule $\log_{10}(10^{-6})$.
2. Simplifique $\ln(a^3/b)$.

3. Resolva $e^{2x} = 7$.
4. Uma relação experimental obedece a $y = Cx^n$. Quando x dobra, y é multiplicado por oito. Determine n .
5. Por que o argumento de um logaritmo físico deve ser adimensional?
6. Quantos decibéis são acrescentados quando a intensidade sonora é multiplicada por 100?
7. Em $N(t) = N_0 e^{-t/\tau}$, determine o instante em que $N = N_0/10$.

1.6 Ângulos e trigonometria

Um ângulo pode ser medido em graus ou radianos. O radiano é a unidade natural do cálculo e relaciona o comprimento de arco s ao raio r :

$$\theta = \frac{s}{r}. \quad (1.85)$$

Uma volta completa corresponde a 2π radianos ou 360° . Portanto,

$$\theta_{\text{rad}} = \theta_{\text{graus}} \frac{\pi}{180}. \quad (1.86)$$

Em um triângulo retângulo,

$$\sin \theta = \frac{\text{cateto oposto}}{\text{hipotenusa}}, \quad \cos \theta = \frac{\text{cateto adjacente}}{\text{hipotenusa}}, \quad (1.87)$$

e

$$\tan \theta = \frac{\sin \theta}{\cos \theta}. \quad (1.88)$$

A identidade fundamental

$$\sin^2 \theta + \cos^2 \theta = 1 \quad (1.89)$$

decorre do teorema de Pitágoras aplicado ao círculo unitário.

Exemplo: componentes de uma força

Uma força de módulo 80 N forma um ângulo de 30° acima da horizontal. Suas componentes cartesianas são

$$F_x = F \cos \theta = 80 \text{ N} \cos 30^\circ \approx 69,3 \text{ N}, \quad (1.90)$$

$$F_y = F \sin \theta = 80 \text{ N} \sin 30^\circ = 40 \text{ N}. \quad (1.91)$$

A [figura 1.2](#) apresenta primeiro o vetor original e, depois, suas projeções sobre os eixos cartesianos. As linhas tracejadas completam o retângulo de decomposição e ajudam a visualizar a soma vetorial $\mathbf{F} = \mathbf{F}_x + \mathbf{F}_y$.

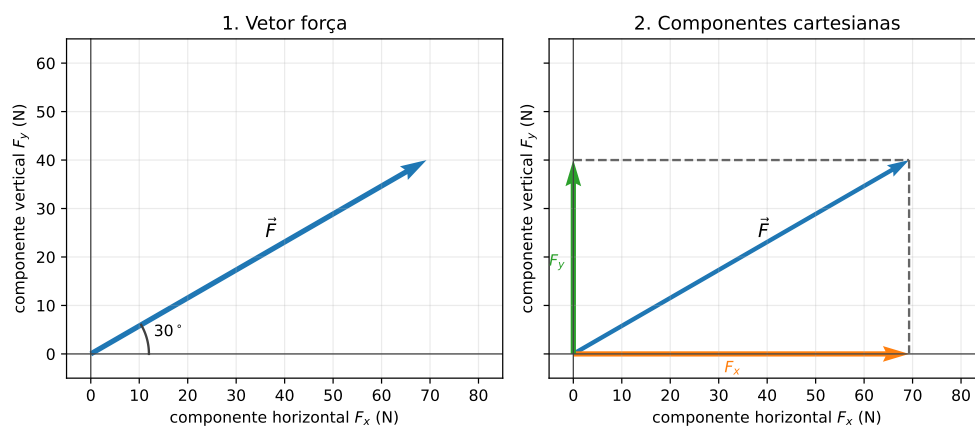


Figura 1.2: Uma força de 80 N a 30° da horizontal e sua decomposição. No segundo painel, $F_x \approx 69,3 \text{ N}$ e $F_y = 40 \text{ N}$.

O programa usado para produzir a figura aplica diretamente as relações calculadas no exemplo. O método `quiver`, introduzido no [seção 0.12](#), recebe a origem da seta e suas componentes horizontal e vertical.

Código 1.7: Representação da força e de suas componentes cartesianas.

```
1  """Representação de uma força e de suas componentes cartesianas."""
2
3  from pathlib import Path
4  import math
5
6  import matplotlib.pyplot as plt
7
8  forca = 80.0 # newtons
9  angulo_graus = 30.0
```

```
10 angulo = math.radians(angulo_graus)
11
12 forca_x = forca * math.cos(angulo)
13 forca_y = forca * math.sin(angulo)
14
15 fig, eixos = plt.subplots(1, 2, figsize=(9, 4.2), sharex=True,
16                             sharey=True)
17
18 for ax in eixos:
19     ax.axhline(0, color="0.25", linewidth=0.8)
20     ax.axvline(0, color="0.25", linewidth=0.8)
21     ax.set_xlim(-5, 85)
22     ax.set_ylim(-5, 65)
23     ax.set_xlabel(r"componente horizontal  $F_x$  (N)")
24     ax.set_ylabel(r"componente vertical  $F_y$  (N)")
25     ax.set_aspect("equal")
26     ax.grid(alpha=0.25)
27
28 # Primeiro painel: somente a força e o ângulo.
29 eixos[0].quiver(
30     0, 0, forca_x, forca_y,
31     angles="xy", scale_units="xy", scale=1,
32     color="tab:blue", width=0.012,
33 )
34 eixos[0].text(forca_x * 0.52, forca_y * 0.60, r" $\vec{F}$ ",
35               fontsize=12)
36 arco_x = [12 * math.cos(math.radians(a)) for a in range(31)]
37 arco_y = [12 * math.sin(math.radians(a)) for a in range(31)]
38 eixos[0].plot(arco_x, arco_y, color="0.25")
39 eixos[0].text(13, 3.5, r" $30^\circ$ ")
40 eixos[0].set_title("1. Vetor força")
41
42 # Segundo painel: força e projeções nos eixos cartesianos.
43 eixos[1].quiver(
44     0, 0, forca_x, forca_y,
45     angles="xy", scale_units="xy", scale=1,
46     color="tab:blue", width=0.010,
47 )
48 eixos[1].quiver(
49     0, 0, forca_x, 0,
50     angles="xy", scale_units="xy", scale=1,
51     color="tab:orange", width=0.012,
52 )
```



```

51 eixos[1].quiver(
52     0, 0, 0, forca_y,
53     angles="xy", scale_units="xy", scale=1,
54     color="tab:green", width=0.012,
55 )
56 eixos[1].plot([forca_x, forca_x], [0, forca_y], "--", color="0.4")
57 eixos[1].plot([0, forca_x], [forca_y, forca_y], "--", color="0.4")
58 eixos[1].text(forca_x * 0.45, -4, r"$F_x$", color="tab:orange")
59 eixos[1].text(-5, forca_y * 0.45, r"$F_y$", color="tab:green")
60 eixos[1].text(forca_x * 0.55, forca_y * 0.60, r"$\vec{F}$",
61               fontsize=12)
62 eixos[1].set_title("2. Componentes cartesianas")
63
64 fig.tight_layout()
65
66 pasta_figuras = Path("figuras") / "cap01"
67 pasta_figuras.mkdir(parents=True, exist_ok=True)
68 arquivo = pasta_figuras / "componentes-forca.pdf"
69 fig.savefig(arquivo, bbox_inches="tight")
70 plt.close(fig)
71
72 print(f"Fx = {forca_x:.1f} N")
73 print(f"Fy = {forca_y:.1f} N")
74 print("Gráfico salvo em figuras/cap01/componentes-forca.pdf")

```

A saída textual esperada é

Código 1.8: Saída esperada do programa de componentes de uma força.

```

Fx = 69.3 N
Fy = 40.0 N
Gráfico salvo em figuras/cap01/componentes-forca.pdf

```

Além de confirmar os valores, o desenho permite verificar geometricamente que $F^2 = F_x^2 + F_y^2$.

1.6.1 Seno e cosseno em Python

As funções `sin` e `cos` do módulo `math` recebem ângulos em radianos. Por isso, o programa converte graus com `math.radians` antes do cálculo. Ele também produz o círculo trigonométrico da [figura 1.3](#) e os gráficos periódicos da [figura 1.4](#).

Código 1.9: Valores de seno e cosseno e geração das figuras trigonométricas.

```
1  """Valores trigonometricos e grafico de seno e cosseno."""
2
3  from pathlib import Path
4  import math
5
6  import matplotlib.pyplot as plt
7  import numpy as np
8
9  for angulo_graus in (0, 30, 45, 60, 90):
10     angulo_rad = math.radians(angulo_graus)
11     seno = math.sin(angulo_rad)
12     cosseno = math.cos(angulo_rad)
13     print(
14         f"{angulo_graus:>2} graus: "
15         f"sen = {seno:.4f}, cos = {cosseno:.4f}"
16     )
17
18  theta = np.linspace(0, 2 * np.pi, 500)
19  angulo_desenho = math.radians(40)
20
21  fig, ax = plt.subplots(figsize=(5, 5))
22  ax.add_patch(plt.Circle((0, 0), 1, fill=False, color="0.25",
23                          linewidth=1.5))
24  x_ponto = math.cos(angulo_desenho)
25  y_ponto = math.sin(angulo_desenho)
26  ax.quiver(
27      0,
28      0,
29      x_ponto,
30      y_ponto,
31      angles="xy",
32      scale_units="xy",
33      scale=1,
34      color="tab:blue",
35      label=r"raio unitario",
36  )
37  ax.plot([x_ponto, x_ponto], [0, y_ponto], "--", color="tab:orange")
38  ax.plot([0, x_ponto], [y_ponto, y_ponto], "--", color="tab:green")
39  ax.text(x_ponto / 2, -0.10, r"$\cos(\theta)$", ha="center")
40  ax.text(x_ponto + 0.08, y_ponto / 2, r"$\sin(\theta)$", va="center")
41
42  arco = np.linspace(0, angulo_desenho, 80)
43  ax.plot(0.22 * np.cos(arco), 0.22 * np.sin(arco), color="tab:red")
```

```

42 ax.text(0.27, 0.08, r"$\theta$")
43 ax.axhline(0, color="0.25", linewidth=0.8)
44 ax.axvline(0, color="0.25", linewidth=0.8)
45 ax.set_xlabel(r"eixo $x$")
46 ax.set_ylabel(r"eixo $y$")
47 ax.set_xlim(-1.2, 1.2)
48 ax.set_ylim(-1.2, 1.2)
49 ax.set_aspect("equal")
50 ax.grid(alpha=0.25)
51 fig.tight_layout()
52
53 pasta_figuras = Path("figuras") / "cap01"
54 pasta_figuras.mkdir(parents=True, exist_ok=True)
55 fig.savefig(pasta_figuras / "circulo-trigonometrico.pdf",
56             bbox_inches="tight")
57 plt.close(fig)
58
59 fig, ax = plt.subplots(figsize=(7, 4.5))
60 ax.plot(theta, np.sin(theta), color="tab:blue", label=r"$\sin(\theta)$")
61 ax.plot(
62     theta,
63     np.cos(theta),
64     color="tab:orange",
65     linestyle="--",
66     label=r"$\cos(\theta)$",
67 )
68 ax.axhline(0, color="0.25", linewidth=0.8)
69 ax.set_xlim(0, 2 * np.pi)
70 ax.set_ylim(-1.15, 1.15)
71 ax.set_xticks(
72     [0, np.pi / 2, np.pi, 3 * np.pi / 2, 2 * np.pi],
73     [r"$0$", r"$\pi/2$", r"$\pi$", r"$3\pi/2$", r"$2\pi$"],
74 )
75 ax.set_xlabel(r"angulo $\theta$ (rad)")
76 ax.set_ylabel("valor da funcao")
77 ax.grid(alpha=0.3)
78 ax.legend()
79 fig.tight_layout()
80 fig.savefig(pasta_figuras / "seno-cosseno.pdf", bbox_inches="tight")
81 plt.close(fig)

```

```
82  
83 print("Figuras salvas em figuras/cap01/")
```

Código 1.10: Saída esperada do programa de trigonometria.

```
0 graus: sen = 0.0000, cos = 1.0000  
30 graus: sen = 0.5000, cos = 0.8660  
45 graus: sen = 0.7071, cos = 0.7071  
60 graus: sen = 0.8660, cos = 0.5000  
90 graus: sen = 1.0000, cos = 0.0000  
Figuras salvas em figuras/cap01/
```

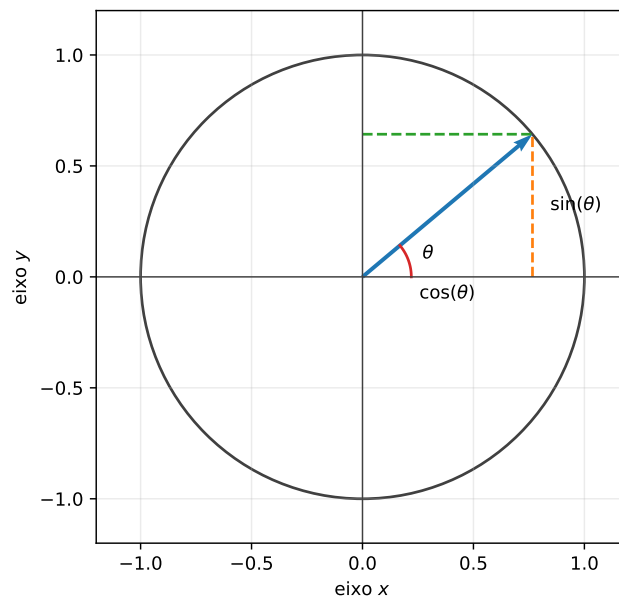


Figura 1.3: No círculo unitário, as coordenadas do ponto são $\cos \theta$ e $\sin \theta$. As linhas tracejadas mostram suas projeções nos eixos.

1.7 Funções e gráficos

Uma função associa cada valor permitido da variável independente a um valor da variável dependente. Em Física, a notação $x(t)$ indica que a posição x depende do tempo t .

Antes de desenhar um gráfico, devemos identificar:

1. as grandezas representadas nos eixos;

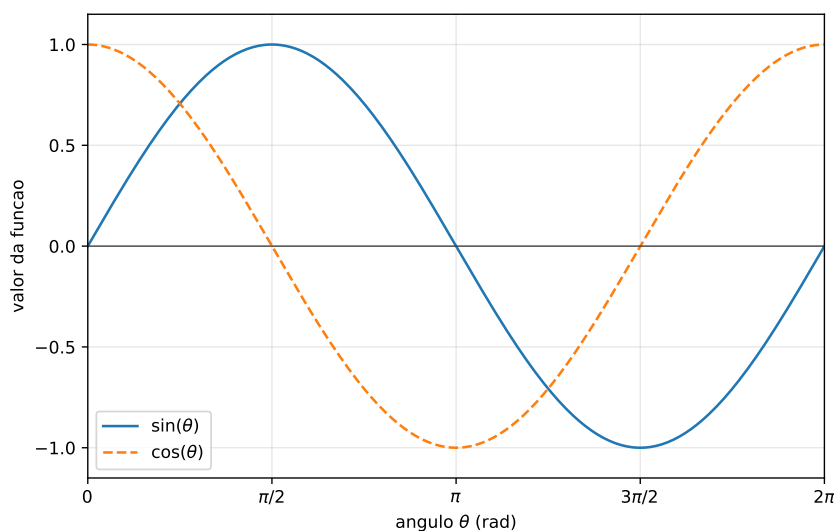


Figura 1.4: Seno e cosseno ao longo de uma volta completa. O estilo tracejado diferencia o cosseno mesmo em impressão sem cores.

2. as unidades utilizadas;
3. o domínio fisicamente relevante;
4. os pontos de interseção com os eixos;
5. o comportamento crescente, decrescente ou oscilatório;
6. os limites do modelo.

1.7.1 Representação de funções em Python

Na [seção 0.12](#), construímos gráficos a partir de listas e arrays. Para representar uma função matemática, seguimos o mesmo processo: escolhemos um domínio amostrado, calculamos a imagem de cada ponto e entregamos os dois arrays ao método `plot`.

O programa a seguir define três funções com `def`. Como as operações usadas por elas são compatíveis com arrays NumPy, uma única chamada, como `funcao_quadratica(x)`, calcula todos os valores necessários. Essa aplicação de uma operação elemento a elemento é chamada de **vetorização**.

Código 1.11: Representação de funções linear, quadrática e inversa.

```
1 """Representação de funções matemáticas em domínios escolhidos."""  
2
```

```

3 from pathlib import Path
4
5 import matplotlib.pyplot as plt
6 import numpy as np
7
8 x = np.linspace(0.2, 4.0, 300)
9
10 def funcao_linear(valor):
11     return 2.0 * valor + 1.0
12
13 def funcao_quadratica(valor):
14     return valor**2
15
16 def funcao_inversa(valor):
17     return 1.0 / valor
18
19 fig, eixos = plt.subplots(1, 3, figsize=(10, 3.4))
20
21 funcoes = [funcao_linear, funcao_quadratica, funcao_inversa]
22 rotulos = [r"$f(x)=2x+1$", r"$g(x)=x^2$", r"$h(x)=1/x$"]
23
24 for ax, funcao, rotulo in zip(eixos, funcoes, rotulos):
25     ax.plot(x, funcao(x), label=rotulo)
26     ax.set_xlabel("x")
27     ax.set_ylabel("valor da função")
28     ax.grid(alpha=0.3)
29     ax.legend()
30
31 fig.tight_layout()
32
33 pasta_figuras = Path("figuras") / "cap01"
34 pasta_figuras.mkdir(parents=True, exist_ok=True)
35 arquivo = pasta_figuras / "funcoes-basicas.pdf"
36 fig.savefig(arquivo, bbox_inches="tight")
37 plt.close(fig)
38
39 print(f"Domínio: {x[0]:.1f} <= x <= {x[-1]:.1f}")
40 print(f"Quantidade de pontos: {x.size}")
41 print("Gráfico salvo em figuras/cap01/funcoes-basicas.pdf")

```

A saída textual esperada é

Código 1.12: Saída esperada do programa de representação de funções.

```
Domínio: 0.2 <= x <= 4.0
```

Quantidade de pontos: 300

Gráfico salvo em figuras/cap01/funcoes-basicas.pdf

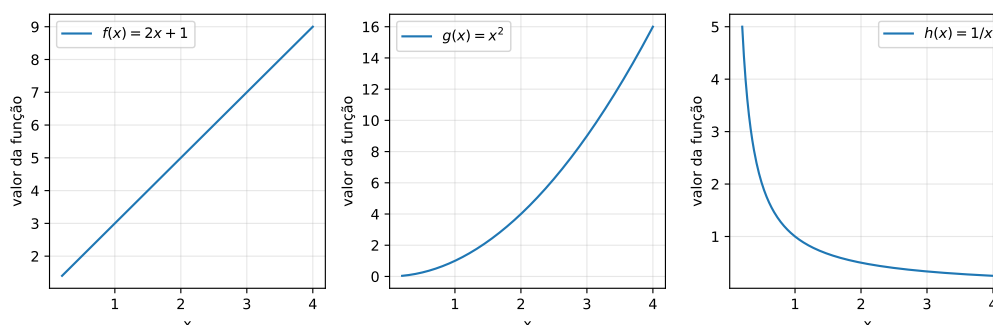


Figura 1.5: Três funções calculadas no domínio amostrado $0,2 \leq x \leq 4,0$. Cada painel possui seus próprios eixos e representa uma relação matemática diferente.

O domínio começa em 0,2, e não em zero, porque a função $1/x$ não está definida em $x = 0$. Essa decisão ilustra uma regra importante: o programa não escolhe automaticamente um domínio matematicamente válido ou fisicamente significativo. Essa responsabilidade pertence a quem constrói o modelo.

No laço **for**, a função **zip** agrupa, em cada repetição, um sistema de eixos, uma função e seu rótulo. Assim, evitamos repetir o mesmo conjunto de comandos para cada painel. A técnica é conveniente aqui porque os três gráficos compartilham a mesma estrutura; ela não deve esconder diferenças importantes entre grandezas, unidades ou domínios.

1.7.2 Relação linear

A função

$$y = mx + b \quad (1.92)$$

representa uma reta. O coeficiente angular

$$m = \frac{\Delta y}{\Delta x} \quad (1.93)$$

mede a variação de y por unidade de x , enquanto b é o valor de y quando $x = 0$.

Na equação (1.1), a inclinação de um gráfico $x \times t$ é a velocidade v , e a interseção com o eixo vertical é a posição inicial x_0 . As unidades da inclinação também carregam significado:

$$[v] = \frac{[x]}{[t]} = \text{m/s}. \quad (1.94)$$

1.7.3 Leis de potência e relações inversas

Funções como $y = kx^2$ aparecem em movimentos acelerados e energias; funções como $y = k/x^2$ aparecem em modelos de campos produzidos por fontes puntiformes. A forma do gráfico ajuda a distinguir essas relações, mas a interpretação deve respeitar o domínio físico. Uma expressão que diverge em $x = 0$ costuma indicar que o modelo idealizado deixou de ser aplicável nessa região.

1.7.4 Comportamento exponencial

Uma função de decaimento possui a forma

$$y(t) = y_0 e^{-t/\tau}, \quad (1.95)$$

onde τ define a escala temporal. Em $t = \tau$,

$$y(\tau) = \frac{y_0}{e} \approx 0,368y_0. \quad (1.96)$$

Esse comportamento surge quando a taxa de variação de uma grandeza é proporcional ao próprio valor da grandeza.

1.8 Derivada: taxa de variação instantânea

A taxa média de variação de f entre x e $x + \Delta x$ é

$$\frac{\Delta f}{\Delta x} = \frac{f(x + \Delta x) - f(x)}{\Delta x}. \quad (1.97)$$

Ao considerar intervalos progressivamente menores, definimos a derivada:

$$\frac{df}{dx} = \lim_{\Delta x \rightarrow 0} \frac{f(x + \Delta x) - f(x)}{\Delta x}. \quad (1.98)$$

Geometricamente, a derivada é a inclinação da reta tangente ao gráfico. Fisicamente, seu significado depende das grandezas envolvidas. Para a posição $x(t)$,

$$v(t) = \frac{dx}{dt}, \quad (1.99)$$

e para a velocidade,

$$a(t) = \frac{dv}{dt} = \frac{d^2x}{dt^2}. \quad (1.100)$$

1.8.1 Derivada analítica e derivada numérica

Para $f(x) = x^2$, a derivada analítica é $f'(x) = 2x$. O programa também estima a derivada numericamente por uma diferença central com intervalo pequeno. A concordância das duas abordagens em $x = 2$ valida o cálculo, enquanto a [figura 1.6](#) mostra a interpretação geométrica da inclinação.

Código 1.13: Comparação entre derivada analítica e diferença central.

```
1  """Derivada como inclinacao da reta tangente."""
2
3  from pathlib import Path
4
5  import matplotlib.pyplot as plt
6  import numpy as np
7
8
9  def f(x):
10     return x**2
11
12
13  x0 = 2.0
14  h = 1e-5
15  derivada_numerica = (f(x0 + h) - f(x0 - h)) / (2 * h)
16  derivada_analitica = 2 * x0
17
18  print(f"Derivada analitica em x = {x0:.1f}: {derivada_analitica:.6f}")
19  print(f"Derivada numerica em x = {x0:.1f}: {derivada_numerica:.6f}")
20
21  x = np.linspace(-0.5, 4.5, 400)
22  tangente = f(x0) + derivada_analitica * (x - x0)
```

```
23
24 fig, ax = plt.subplots(figsize=(7, 4.5))
25 ax.plot(x, f(x), color="tab:blue", linewidth=2, label=r"$f(x)=x^2$")
26
27 ax.plot(
28     x,
29     tangente,
30     color="tab:orange",
31     linestyle="--",
32     label="reta tangente em x = 2",
33 )
34 ax.scatter([x0], [f(x0)], color="tab:red", zorder=3, label="ponto
35     de tangencia")
36 ax.set_xlabel(r"$x$")
37 ax.set_ylabel(r"$f(x)$")
38 ax.set_ylim(-2, 20)
39 ax.grid(alpha=0.3)
40 ax.legend()
41 fig.tight_layout()
42
43 pasta_figuras = Path("figuras") / "cap01"
44 pasta_figuras.mkdir(parents=True, exist_ok=True)
45 arquivo = pasta_figuras / "derivada-tangente.pdf"
46 fig.savefig(arquivo, bbox_inches="tight")
47 plt.close(fig)
48
49 print("Gráfico salvo em figuras/cap01/derivada-tangente.pdf")
```

Código 1.14: Saída esperada do programa de derivadas.

```
Derivada analitica em x = 2.0: 4.000000
Derivada numerica em x = 2.0: 4.000000
Gráfico salvo em figuras/cap01/derivada-tangente.pdf
```

1.8.2 Regras de derivação mais utilizadas

Sejam $u(x)$ e $v(x)$ funções diferenciáveis e c uma constante. As regras seguintes serão usadas repetidamente ao longo do livro.

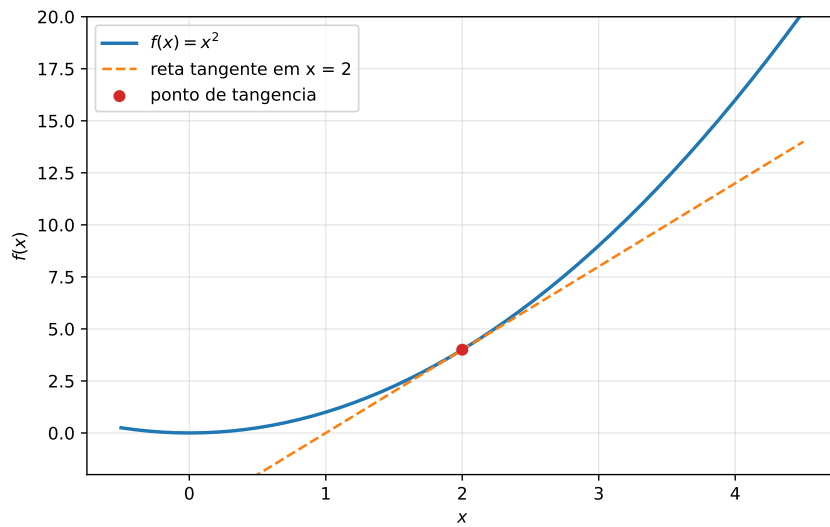


Figura 1.6: A derivada de $f(x) = x^2$ em $x = 2$ é a inclinação da reta tangente no ponto $(2, 4)$.

Constante, multiplicação por constante e soma

$$\frac{d}{dx}c = 0, \quad (1.101)$$

$$\frac{d}{dx}[cu(x)] = cu'(x), \quad (1.102)$$

$$\frac{d}{dx}[u(x) \pm v(x)] = u'(x) \pm v'(x). \quad (1.103)$$

Essas regras permitem derivar uma expressão termo a termo. Por exemplo,

$$\frac{d}{dt}\left(x_0 + v_0t + \frac{1}{2}at^2\right) = v_0 + at. \quad (1.104)$$

Produto e quociente

$$\frac{d}{dx}[u(x)v(x)] = u'(x)v(x) + u(x)v'(x), \quad (1.105)$$

$$\frac{d}{dx}\left[\frac{u(x)}{v(x)}\right] = \frac{u'(x)v(x) - u(x)v'(x)}{[v(x)]^2}, \quad v(x) \neq 0. \quad (1.106)$$

Não se deve derivar um produto multiplicando as derivadas, nem um quociente dividindo-as. As expressões completas acima são necessárias.

Regra da cadeia

Quando uma função está dentro de outra, usamos

$$\frac{d}{dx}f(u(x)) = f'(u(x))u'(x). \quad (1.107)$$

Derivamos a função externa e multiplicamos pela derivada da função interna. Dois exemplos importantes em Física são

$$\frac{d}{dt}\cos(\omega t + \phi) = -\omega \sin(\omega t + \phi), \quad (1.108)$$

$$\frac{d}{dt}e^{-t/\tau} = -\frac{1}{\tau}e^{-t/\tau}. \quad (1.109)$$

Derivadas elementares

$$\frac{d}{dx}x^n = nx^{n-1}, \quad (1.110)$$

$$\frac{d}{dx}\sin x = \cos x, \quad (1.111)$$

$$\frac{d}{dx}\cos x = -\sin x, \quad (1.112)$$

$$\frac{d}{dx}e^x = e^x, \quad (1.113)$$

$$\frac{d}{dx}\ln|x| = \frac{1}{x}, \quad x \neq 0. \quad (1.114)$$

As fórmulas trigonométricas pressupõem que o argumento esteja em radianos. Regras específicas para outras funções e derivadas implícitas serão introduzidas quando forem necessárias.

Exemplo: movimento não uniforme

Considere

$$x(t) = 2 \text{ m} + (3 \text{ m/s})t + (0,5 \text{ m/s}^2)t^2. \quad (1.115)$$

Derivando em relação ao tempo,

$$v(t) = 3 \text{ m/s} + (1 \text{ m/s}^2)t. \quad (1.116)$$

A aceleração é constante e vale 1 m/s^2 . Observe que os coeficientes possuem unidades diferentes para que todos os termos de $x(t)$ tenham dimensão de comprimento.

1.9 Integral: acumulação

A integral definida pode ser entendida como o limite de uma soma de pequenas contribuições. Se conhecemos a velocidade em função do tempo, o deslocamento entre t_1 e t_2 é

$$\Delta x = \int_{t_1}^{t_2} v(t) dt. \quad (1.117)$$

Geometricamente, essa integral corresponde à área algébrica sob o gráfico $v \times t$. Regiões abaixo do eixo temporal contribuem negativamente, o que distingue deslocamento de distância total percorrida.

1.9.1 Integral analítica e integração numérica

Para $v(t) = 2t$, a integral entre 0 e 4 s vale 16 m. O programa compara esse resultado com a regra dos trapézios implementada por `numpy.trapezoid`. A região sombreada na [figura 1.7](#) representa o deslocamento acumulado.

Código 1.15: Comparação entre integral analítica e integração numérica.

```

1  """Integral como area algebrica sob um grafico."""
2
3  from pathlib import Path
4
5  import matplotlib.pyplot as plt
6  import numpy as np
7
8  tempo = np.linspace(0, 4, 401) # s
9  velocidade = 2 * tempo # m/s
10
11 deslocamento_numerico = np.trapezoid(velocidade, tempo)
12 deslocamento_exato = 4**2
13
14 print(f"Deslocamento exato: {deslocamento_exato:.2f} m")
15 print(f"Deslocamento numerico: {deslocamento_numerico:.2f} m")
16
17 fig, ax = plt.subplots(figsize=(7, 4.5))
18 ax.plot(tempo, velocidade, color="tab:blue", linewidth=2, label=r"$v(t)=2t$")
19 ax.fill_between(
20     tempo,
21     velocidade,
22     color="tab:blue",

```

```
23     alpha=0.2,  
24     label=r"$\Delta x=\int_0^4 v(t)\,dt$",  
25 )  
26 ax.set_xlabel("tempo, t (s)")  
27 ax.set_ylabel("velocidade, v (m/s)")  
28 ax.set_xlim(0, 4)  
29 ax.set_ylim(0, 8.5)  
30 ax.grid(alpha=0.3)  
31 ax.legend()  
32 fig.tight_layout()  
33  
34 pasta_figuras = Path("figuras") / "cap01"  
35 pasta_figuras.mkdir(parents=True, exist_ok=True)  
36 arquivo = pasta_figuras / "integral-area.pdf"  
37 fig.savefig(arquivo, bbox_inches="tight")  
38 plt.close(fig)  
39  
40 print("Gráfico salvo em figuras/cap01/integral-area.pdf")
```

Código 1.16: Saída esperada do programa de integrais.

Deslocamento exato: 16.00 m

Deslocamento numerico: 16.00 m

Gráfico salvo em figuras/cap01/integral-area.pdf

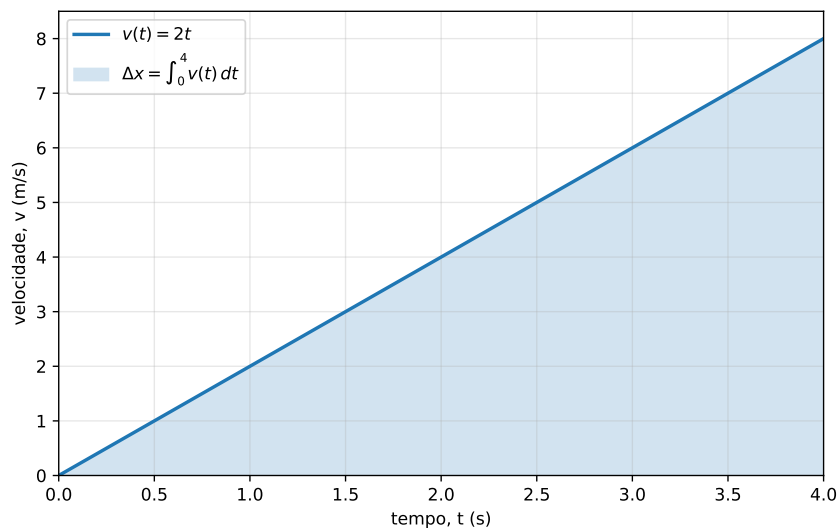


Figura 1.7: O deslocamento é a área sob o gráfico de velocidade por tempo. Para $v(t) = 2t$, a área triangular vale 16 m.

1.9.2 Regras de integração mais utilizadas

Uma antiderivada de $f(x)$ é uma função $F(x)$ cuja derivada é $f(x)$. Escrevemos

$$\int f(x) \, dx = F(x) + C, \quad (1.118)$$

onde C é a constante de integração. Ela aparece porque funções que diferem apenas por uma constante possuem a mesma derivada.

Linearidade

Se c é constante,

$$\int c f(x) \, dx = c \int f(x) \, dx, \quad (1.119)$$

$$\int [f(x) \pm g(x)] \, dx = \int f(x) \, dx \pm \int g(x) \, dx. \quad (1.120)$$

Assim como na derivação, uma soma pode ser tratada termo a termo. Entretanto, não existe uma regra geral que permita integrar um produto apenas multiplicando integrais.

Integrais elementares

$$\int x^n \, dx = \frac{x^{n+1}}{n+1} + C, \quad n \neq -1, \quad (1.121)$$

$$\int \frac{1}{x} \, dx = \ln |x| + C, \quad x \neq 0, \quad (1.122)$$

$$\int e^{ax} \, dx = \frac{1}{a} e^{ax} + C, \quad a \neq 0, \quad (1.123)$$

$$\int \sin(ax) \, dx = -\frac{1}{a} \cos(ax) + C, \quad a \neq 0, \quad (1.124)$$

$$\int \cos(ax) \, dx = \frac{1}{a} \sin(ax) + C, \quad a \neq 0. \quad (1.125)$$

Os fatores $1/a$ aparecem por causa da regra da cadeia. Podemos verificar cada resultado derivando o lado direito.

Integral definida e teorema fundamental

Se $F'(x) = f(x)$, então

$$\int_a^b f(x) \, dx = F(b) - F(a). \quad (1.126)$$

Em uma integral definida, a constante C não aparece no resultado porque seria cancelada na diferença entre os limites. O valor representa uma acumulação líquida: contribuições positivas e negativas são combinadas algebricamente.

Integração por substituição, integração por partes e outras técnicas serão apresentadas apenas quando os modelos físicos exigirem seu uso.

Exemplo: deslocamento com velocidade variável

Para $v(t) = \alpha t$, onde α possui unidade de aceleração,

$$\Delta x = \int_0^T \alpha t \, dt = \frac{1}{2} \alpha T^2. \quad (1.127)$$

O mesmo resultado corresponde à área de um triângulo de base T e altura αT no gráfico de velocidade por tempo.

1.10 Máximos, mínimos e interpretação física

Em um ponto interior onde uma função diferenciável possui máximo ou mínimo local, sua derivada é nula:

$$f'(x) = 0. \quad (1.128)$$

Essa condição identifica pontos candidatos, mas não os classifica. Se

$$f''(x) > 0, \quad (1.129)$$

a concavidade é voltada para cima e o ponto é um mínimo local. Se $f''(x) < 0$, temos um máximo local. Também é necessário examinar fronteiras do domínio e pontos onde a derivada não existe.

Exemplo: altura máxima

Uma pequena esfera é lançada verticalmente, e sua altura é descrita pelo modelo

$$h(t) = 1,2 \text{ m} + (8 \text{ m/s}) t - \frac{1}{2} (9,8 \text{ m/s}^2) t^2. \quad (1.130)$$

O instante de altura máxima satisfaz $dh/dt = 0$:

$$8 - 9,8t = 0 \quad \Rightarrow \quad t \approx 0,816 \text{ s}. \quad (1.131)$$

Como $d^2h/dt^2 = -9,8 \text{ m/s}^2$, a concavidade é negativa e o ponto é de máximo. A interpretação física coincide com o cálculo: no ponto mais alto, a velocidade vertical é momentaneamente nula.

1.11 Estratégia para resolver problemas

Uma solução organizada costuma seguir estas etapas:

1. identificar a pergunta e o sistema físico;
2. listar dados, incógnitas e unidades;
3. desenhar uma representação quando ela ajudar;
4. declarar hipóteses e escolher um modelo;
5. desenvolver a solução simbolicamente antes de substituir números;
6. verificar dimensões, sinal e ordem de grandeza;
7. interpretar o resultado no contexto do problema;
8. reconhecer limitações do modelo.

Programas de computador automatizam operações e ampliam nossa capacidade de exploração. Entretanto, não escolhem sozinhos o sistema, as hipóteses nem o significado físico do resultado. Essa responsabilidade permanece com quem modela.

1.12 Síntese

Neste capítulo:

- usamos notação científica e potências para lidar com escalas físicas;
- distinguimos proporcionalidade direta, quadrática e inversa;
- revisamos logaritmos e trigonometria;
- interpretamos gráficos como representações de relações entre grandezas;
- associamos derivadas a taxas instantâneas;
- associamos integrais a acumulações e áreas algébricas;
- usamos derivadas para localizar extremos;
- estabelecemos critérios para verificar resultados físicos.

1.13 Exercícios propostos

Os exercícios abaixo foram elaborados para este livro.

1. Escreva em notação científica o diâmetro de $0,000\,000\,32\text{ m}$. Em seguida, converta-o para nanômetros.
2. A energia armazenada em certo sistema é proporcional ao quadrado de uma variável q . Por qual fator a energia muda quando q é multiplicada por 3?
3. A intensidade de um sinal idealizado é inversamente proporcional ao quadrado da distância. Compare a intensidade a 2 m e a 6 m da fonte.
4. Uma força de 120 N forma 40° com o eixo horizontal. Determine suas componentes cartesianas.
5. Converta 150° para radianos e $5\pi/6$ radianos para graus.
6. Para $x(t) = 4 + 2t - 3t^2$, determine as funções velocidade e aceleração. Indique as unidades que os coeficientes devem possuir se x estiver em metros e t em segundos.

7. A velocidade de um objeto é $v(t) = 5 - 2t$, em unidades do SI. Calcule o deslocamento entre $t = 0$ e $t = 4$ s. Explique o sinal do resultado usando o gráfico $v \times t$.
8. Determine os pontos críticos de $f(x) = x^3 - 6x^2 + 9x + 2$ e classifique-os quando possível.
9. A temperatura de um corpo é modelada por $T(t) = T_{\text{amb}} + \Delta T_0 e^{-t/\tau}$. Mostre que a taxa de variação de $T - T_{\text{amb}}$ é proporcional ao próprio excesso de temperatura.
10. Uma reta em um gráfico posição-tempo passa pelos pontos (2 s, 7 m) e (8 s, 25 m). Determine sua inclinação, interprete-a fisicamente e encontre a posição em $t = 0$.

Resoluções das atividades propostas

As resoluções são organizadas por capítulo e mantêm a numeração dos exercícios no texto principal. Em cada solução, o desenvolvimento matemático precede a substituição numérica. O Python é usado como verificação quando acrescenta valor ao raciocínio.

Capítulo 1 — Ferramentas matemáticas para a Física

Exercício 1

Para escrever o diâmetro em notação científica, deslocamos a vírgula sete posições para a direita:

$$0,000\,000\,32\text{ m} = 3,2 \cdot 10^{-7}\text{ m}. \quad (\text{S.1})$$

Como $1\text{ m} = 1 \cdot 10^9\text{ nm}$, multiplicamos o valor em metros por 10^9 :

$$(3,2 \times 10^{-7}\text{ m}) \left(10^9 \frac{\text{nm}}{\text{m}}\right) = 3,2 \times 10^2\text{ nm} = 320\text{ nm}. \quad (\text{S.2})$$

Portanto, o diâmetro é $3,2 \cdot 10^{-7}\text{ m} = 320\text{ nm}$.

Exercício 2

Se a energia E é proporcional a q^2 , podemos escrever $E = Cq^2$, onde C permanece constante. Ao substituir q por $3q$, obtemos

$$E' = C(3q)^2 = 9Cq^2 = 9E. \quad (\text{S.3})$$

Logo, a energia é 9 vezes maior.

Exercício 3

Para uma intensidade I inversamente proporcional a r^2 , temos $I = C/r^2$. Assim,

$$\frac{I(2\text{ m})}{I(6\text{ m})} = \frac{C/2^2}{C/6^2} = \frac{6^2}{2^2} = 9. \quad (\text{S.4})$$

A intensidade a 2 m é, portanto, 9 vezes a intensidade a 6 m. Equivalentemente, $I(6\text{ m}) = I(2\text{ m})/9$.

Exercício 4

Tomando o ângulo a partir do eixo horizontal,

$$F_x = F \cos \theta = 120\text{ N} \cos 40^\circ \approx 91,9\text{ N}, \quad (\text{S.5})$$

$$F_y = F \sin \theta = 120\text{ N} \sin 40^\circ \approx 77,1\text{ N}. \quad (\text{S.6})$$

As duas componentes são positivas porque o vetor está no primeiro quadrante. Portanto,

$$\mathbf{F} \approx (91,9\text{ N})\hat{\mathbf{i}} + (77,1\text{ N})\hat{\mathbf{j}}. \quad (\text{S.7})$$

Como verificação, $\sqrt{F_x^2 + F_y^2} \approx 120\text{ N}$.

Exercício 5

Para converter graus em radianos, multiplicamos por $\pi/180^\circ$:

$$150^\circ \frac{\pi\text{ rad}}{180^\circ} = \left[\frac{5\pi}{6} \text{ rad} \right]. \quad (\text{S.8})$$

No sentido inverso,

$$\frac{5\pi}{6} \text{ rad} \frac{180^\circ}{\pi\text{ rad}} = \left[150^\circ \right]. \quad (\text{S.9})$$

Os resultados são consistentes porque as duas expressões representam o mesmo ângulo.

Exercício 6

A velocidade é a derivada da posição em relação ao tempo:

$$v(t) = \frac{dx}{dt} = 2 - 6t. \quad (\text{S.10})$$

A aceleração é a derivada da velocidade:

$$a(t) = \frac{dv}{dt} = -6. \quad (\text{S.11})$$

Se x é medido em metros e t em segundos, cada termo de $x(t)$ precisa ter unidade de comprimento. Portanto, o termo constante 4 tem unidade de metro, o coeficiente 2 tem unidade de m/s, e o coeficiente -3 tem unidade de m/s². Assim,

$$v(t) = \boxed{(2 - 6t) \text{ m/s}}, \quad (\text{S.12})$$

$$a(t) = \boxed{-6 \text{ m/s}^2}. \quad (\text{S.13})$$

Na primeira expressão, os números carregam as unidades necessárias para que ambos os termos representem velocidade.

Exercício 7

O deslocamento é a integral da velocidade:

$$\Delta x = \int_0^4 (5 - 2t) dt \quad (\text{S.14})$$

$$= [5t - t^2]_0^4 \quad (\text{S.15})$$

$$= 20 - 16 \quad (\text{S.16})$$

$$= \boxed{4 \text{ m}}. \quad (\text{S.17})$$

No gráfico $v \times t$, a velocidade se anula em $t = 2,5$ s. Entre 0 e 2,5 s, a área triangular positiva vale

$$A_+ = \frac{1}{2}(2,5 \text{ s})(5 \text{ m/s}) = 6,25 \text{ m}. \quad (\text{S.18})$$

Entre 2,5 s e 4 s, a região fica abaixo do eixo e contribui com

$$A_- = -\frac{1}{2}(1,5 \text{ s})(3 \text{ m/s}) = -2,25 \text{ m}. \quad (\text{S.19})$$

Logo, $\Delta x = A_+ + A_- = 4 \text{ m}$. O sinal positivo indica que, apesar da inversão do movimento após 2,5 s, o deslocamento no sentido positivo foi maior.

Exercício 8

Os pontos críticos de uma função diferenciável ocorrem onde $f'(x) = 0$. Para

$$f(x) = x^3 - 6x^2 + 9x + 2, \quad (\text{S.20})$$

temos

$$f'(x) = 3x^2 - 12x + 9 = 3(x - 1)(x - 3). \quad (\text{S.21})$$

Portanto, os valores críticos são $x = 1$ e $x = 3$. A segunda derivada é

$$f''(x) = 6x - 12. \quad (\text{S.22})$$

Em $x = 1$, $f''(1) = -6 < 0$, logo há um máximo local. Como $f(1) = 6$, o ponto é $\boxed{(1, 6)}$.

Em $x = 3$, $f''(3) = 6 > 0$, logo há um mínimo local. Como $f(3) = 2$, o ponto é $\boxed{(3, 2)}$.

Exercício 9

Definimos o excesso de temperatura em relação ao ambiente como

$$\Theta(t) = T(t) - T_{\text{amb}} = \Delta T_0 e^{-t/\tau}. \quad (\text{S.23})$$

Derivando em relação ao tempo,

$$\frac{d\Theta}{dt} = -\frac{\Delta T_0}{\tau} e^{-t/\tau} \quad (\text{S.24})$$

$$= -\frac{1}{\tau} \Theta. \quad (\text{S.25})$$

Como $\Theta = T - T_{\text{amb}}$, concluímos que

$$\boxed{\frac{d}{dt}(T - T_{\text{amb}}) = -\frac{1}{\tau}(T - T_{\text{amb}})}. \quad (\text{S.26})$$

A taxa é proporcional ao próprio excesso de temperatura. O sinal negativo indica que, para um corpo inicialmente mais quente que o ambiente, o excesso diminui com o tempo.

Exercício 10

A inclinação da reta posição-tempo é

$$v = \frac{\Delta x}{\Delta t} = \frac{25 \text{ m} - 7 \text{ m}}{8 \text{ s} - 2 \text{ s}} = \boxed{3 \text{ m/s}}. \quad (\text{S.27})$$

Essa inclinação representa a velocidade constante. Escrevendo a posição como $x(t) = x_0 + vt$ e usando o ponto (2 s, 7 m), obtemos

$$7 \text{ m} = x_0 + (3 \text{ m/s})(2 \text{ s}), \quad (\text{S.28})$$

de onde

$$\boxed{x_0 = 1 \text{ m}}. \quad (\text{S.29})$$

Portanto, a equação da posição é $\boxed{x(t) = 1 + 3t}$, com x em metros e t em segundos.

Verificação numérica em Python

O programa a seguir confere os resultados numéricos dos exercícios 1, 4, 5, 7, 8 e 10. Ele não substitui os desenvolvimentos anteriores: apenas executa as operações depois que o modelo já foi estabelecido.

Código S.1: Verificação numérica de resultados selecionados do Capítulo 1.

```

1  """Verificação numérica de exercícios selecionados do Capítulo 1.
   """
2
3  import math
4
5  # Exercício 1
6  diametro_m = 0.00000032
7  diametro_nm = diametro_m / 1e-9
8
9  # Exercício 4
10 forca = 120.0
11 angulo = math.radians(40.0)
12 forca_x = forca * math.cos(angulo)
13 forca_y = forca * math.sin(angulo)
14
15 # Exercício 5
16 angulo_rad = math.radians(150.0)
17 angulo_graus = math.degrees(5 * math.pi / 6)

```

```
18
19 # Exercício 7
20 deslocamento = (5 * 4 - 4**2) - (5 * 0 - 0**2)
21
22 # Exercício 8
23 def f(x):
24     return x**3 - 6 * x**2 + 9 * x + 2
25
26 # Exercício 10
27 velocidade = (25 - 7) / (8 - 2)
28 posicao_inicial = 7 - velocidade * 2
29
30 print(f"Ex. 1: {diametro_m:.1e} m = {diametro_nm:.0f} nm")
31 print(f"Ex. 4: Fx = {forca_x:.1f} N; Fy = {forca_y:.1f} N")
32 print(f"Ex. 5: 150 graus = {angulo_rad / math.pi:.3f} pi rad")
33 print(f"Ex. 5: 5pi/6 rad = {angulo_graus:.0f} graus")
34 print(f"Ex. 7: deslocamento = {deslocamento:.1f} m")
35 print(f"Ex. 8: f(1) = {f(1):.1f}; f(3) = {f(3):.1f}")
36 print(f"Ex. 10: v = {velocidade:.1f} m/s; x0 = {posicao_inicial:.1f} m")
```

A saída esperada é

Código S.2: Saída esperada da verificação das soluções do Capítulo 1.

```
Ex. 1: 3.2e-07 m = 320 nm
Ex. 4: Fx = 91.9 N; Fy = 77.1 N
Ex. 5: 150 graus = 0.833 pi rad
Ex. 5: 5pi/6 rad = 150 graus
Ex. 7: deslocamento = 4.0 m
Ex. 8: f(1) = 6.0; f(3) = 2.0
Ex. 10: v = 3.0 m/s; x0 = 1.0 m
```